

SKALA

Git 이해/활용

2026.07

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.



1. 프로그래밍 개요

2. 개발 환경 구성

3. Git

4. GitHub

1. 프로그래밍 개요

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

What is Program?

1. As a verb, to program a computer is writing instructions, statements, or commands that instruct the computer how to process data. There are several programming languages used to program a computer. Using one of these programming languages a programmer can write code to instruct the computer how to perform different tasks. For example, many programmers created the web browser you're using to connect to the Internet and download and read web pages like this page.
2. As a noun, a program, also called an application or software, is a set of instructions that process input, manipulate data, and output a result. For example, Microsoft Word is a word processing program that allows users to create and write documents. A browser, like the one you're using to view this page, is a program.

How Programming Works

- **Problem definition:** Clearly define the problem you want to solve and what you want the program to achieve.
- **Algorithm design:** Develop a step-by-step procedure for solving the problem.
- **Coding:** Translate the algorithm into a programming language using a text editor or integrated development environment (IDE).
- **Testing and debugging:** Run the program and identify and fix any errors.
- **Deployment:** Share the program with others or use it for your own purposes.

Which Programming Language should you choose as your First Language?

Language	Strengths	Weaknesses
C++	Well known for high performance and efficiency, system programming, game development, embedded systems	Steeper learning curve, potential for complex syntax
Java	Widely used for enterprise apps, Android dev, big data	Object-oriented concepts can be complex for beginners
Python	Easy to learn, versatile, widely used for web dev, data sci, AI	Can be slower than compiled languages
JavaScript	Essential for web dev, interactive elements, front-end apps	Challenging to debug complex code
C#	Great for game dev, desktop apps, web services	Requires more upfront learning compared to others
Ruby	Object-oriented, popular for web dev with frameworks like Rails	Rails framework can be overwhelming for beginners

Setting Up Your Development Environment

Choose a Text Editor or IDE:

- Text Editors: Sublime Text, Atom, Notepad++ (lightweight, good for beginners)
- IDEs: Visual Studio Code, PyCharm, IntelliJ IDEA (feature-rich, recommended for larger projects)

Install a Compiler or Interpreter:

- Compilers: Convert code to machine language (C++, Java)
- Interpreters: Execute code line by line (Python, JavaScript)

Variables and Data Types

Understanding Variable Declaration and Usage:

- Variables are containers that hold data and can be assigned different values during program execution.
- To declare a variable, you specify its name and data type, followed by an optional assignment statement.
- Example: `age = 25` (declares a variable named `age` of type integer and assigns it the value 25).
- Variables can be reassigned new values throughout the program.

Exploring Different Data Types:

- **Integers:** Whole numbers without decimal points (e.g., 1, 2, -3).
- **Floats:** Decimal numbers with a fractional part (e.g., 3.14, 10.5).
- **Booleans:** True or False values used for conditions.
- **Characters:** Single letters or symbols ('a', '\$', '#').
- **Strings:** Sequences of characters ("Hello, world!").
- **Other data types:** Arrays, lists, dictionaries, etc. (depending on the language).

Variables and Data Types

Operations with Different Data Types:

Each data type has supported operations.

- **Arithmetic operators** (+, -, *, /) work with integers and floats.
- **Comparison operators** (==, !=, >, <, >=, <=) compare values.
- **Logical operators** (&&, ||, !) combine conditions.
- **Concatenation** (+) joins strings.
- Operations with incompatible data types may lead to errors.

Operators and Expressions

Arithmetic Operators:

- Perform basic mathematical calculations (+, -, *, /, %, **, //).
- % (modulo) returns the remainder after division.
- ** (power) raises a number to a certain power.
- // (floor division) discards the fractional part of the result.

Comparison Operators:

- Evaluate conditions and return True or False.
- == (equal), != (not equal), > (greater than), < (less than), >= (greater than or equal), <= (less than or equal).

Operators and Expressions

Logical Operators: Combine conditions and produce True or False.

- **&& (and):** both conditions must be True.
- **|| (or):** at least one condition must be True.
- **! (not):** reverses the truth value of a condition.

Building Expressions:

- Combine variables, operators, and constants to form expressions.
- Expressions evaluate to a single value.
Example: `result = age + 10 * 2` (calculates the sum of age and 20).

Control Flow Statements

Conditional Statements: Control the flow of execution based on conditions.

- **if-else:** Executes one block of code if the condition is True and another if it's False.
- **switch-case:** Executes different code blocks depending on the value of a variable.

Looping Statements: Repeat a block of code multiple times.

- **for:** Executes a block a specific number of times.
- **while:** Executes a block while a condition is True.
- **do-while:** Executes a block at least once and then repeats while a condition is True.

Functions

Defining and Calling Functions:

- Blocks of code that perform a specific task.
- Defined with a function name, parameters (optional), and a code block.
- Called throughout the program to execute the defined functionality.

Passing Arguments to Functions:

- Values passed to functions for processing.

Returning Values from Functions:

- Functions can return a value after execution.
- Useful for collecting results.

Recursion:

- A function calling itself with a modified input.
- Useful for solving problems that involve repetitive tasks with smaller inputs.

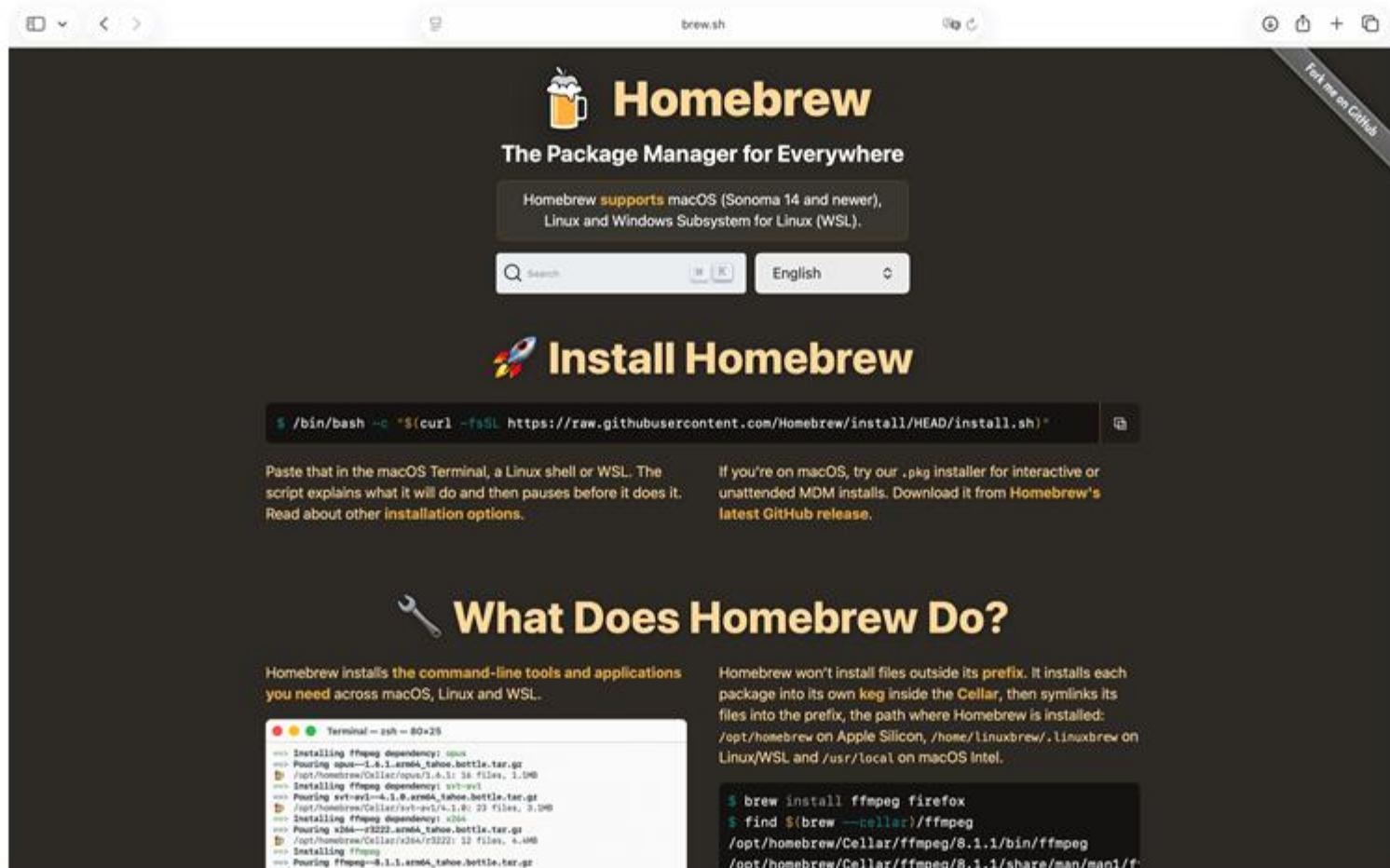
2. 개발 환경 구성

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Homebrew 소개

Package Manager

개발 환경 구성



The screenshot shows the Homebrew website with a dark theme. At the top, the Homebrew logo (a beer mug) is next to the title "Homebrew". Below it, the tagline "The Package Manager for Everywhere" is displayed. A box indicates that Homebrew supports macOS (Sonoma 14 and newer), Linux, and Windows Subsystem for Linux (WSL). There is a search bar and a language selector set to "English".

The main section is titled "Install Homebrew" with a rocket icon. It provides a terminal command to install Homebrew: `$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`. Below the command, it explains that the script explains what it will do and then pauses before it does it, and provides a link to read about other installation options. It also mentions that if you're on macOS, you can try the .pkg installer for interactive or unattended MDM installs, and to download it from Homebrew's latest GitHub release.

The next section is titled "What Does Homebrew Do?" with a wrench icon. It explains that Homebrew installs the command-line tools and applications you need across macOS, Linux, and WSL. It also states that Homebrew won't install files outside its prefix, and it installs each package into its own keg inside the Cellar, then symlinks its files into the prefix, the path where Homebrew is installed. The paths for Apple Silicon, Linuxbrew, and macOS Intel are listed.

At the bottom left, there is a terminal window showing the output of the installation script. It shows the installation of the ffmpeg dependency, the installation of the ffmpeg package, and the installation of the ffmpeg command-line tool. The output is as follows:

```
Terminal - zsh - 80x25
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
Installing ffmpeg dependency: opus
Pouring opus--1.4.1.arm64_tahoe.bottle.tar.gz
/opt/homebrew/Cellar/opus/1.4.1: 16 files, 1.0MB
Installing ffmpeg dependency: x264
Pouring x264--0.164.arm64_tahoe.bottle.tar.gz
/opt/homebrew/Cellar/x264/0.164: 23 files, 3.0MB
Installing ffmpeg dependency: x265
Pouring x265--3.222.arm64_tahoe.bottle.tar.gz
/opt/homebrew/Cellar/x265/3.222: 12 files, 4.4MB
Installing ffmpeg
Pouring ffmpeg--8.1.1.arm64_tahoe.bottle.tar.gz
```

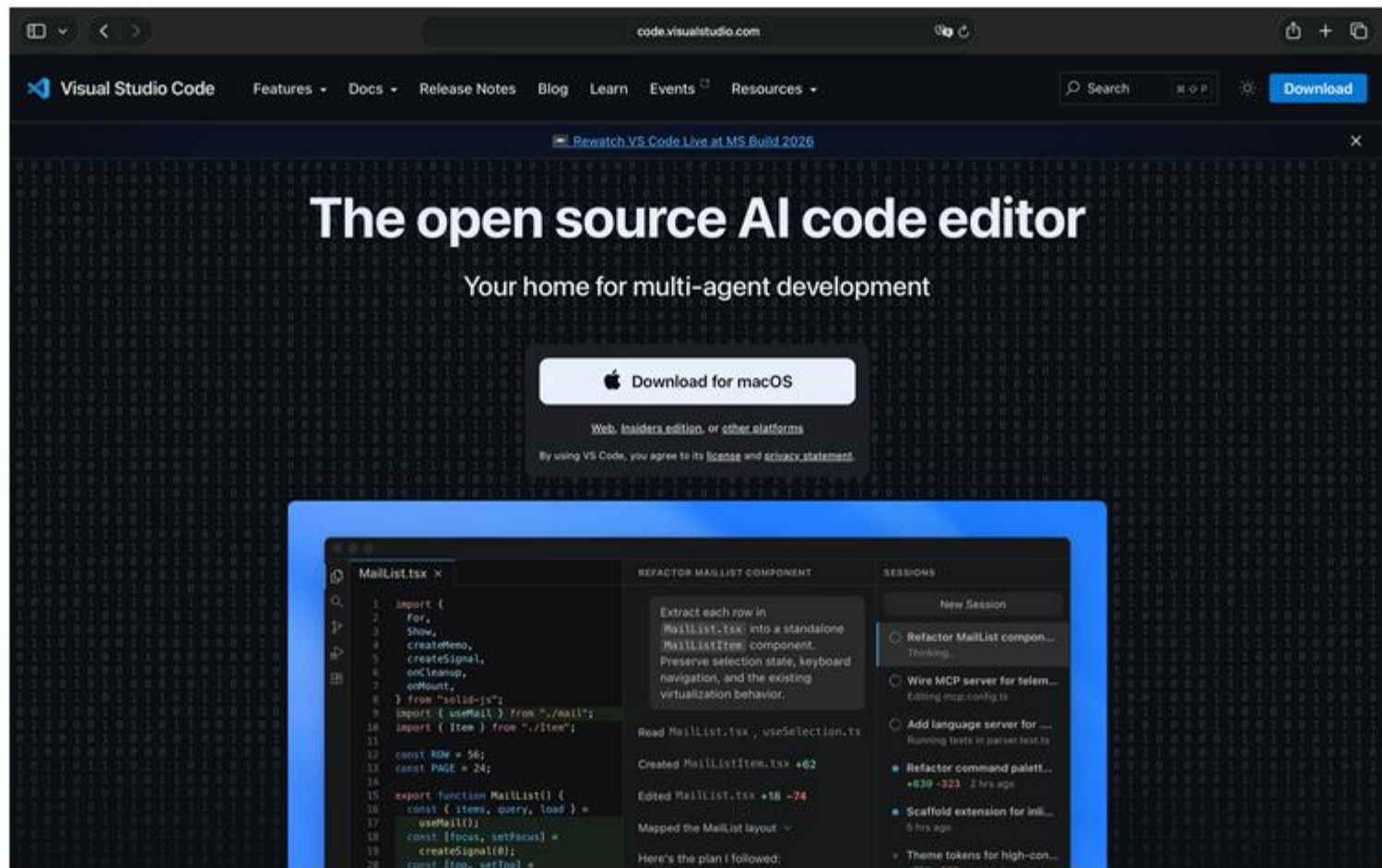
At the bottom right, there is a terminal window showing the output of the `brew install ffmpeg` command. It shows the installation of the ffmpeg package and the creation of the ffmpeg command-line tool. The output is as follows:

```
$ brew install ffmpeg
$ find $(brew --cellar)/ffmpeg
/opt/homebrew/Cellar/ffmpeg/8.1.1/bin/ffmpeg
/opt/homebrew/Cellar/ffmpeg/8.1.1/share/man/man1/f
```

Visual Studio Code 소개

Open Source Code Editor

개발 환경 구성



Visual Studio Code

Keyboard shortcuts for macOS

General

⌘P, F1	Show Command Palette
⌘P	Quick Open, Go to File...
⌘N	New window/instance
⌘W	Close window/instance
⌘L	User Settings
⌘K ⌘S	Keyboard Shortcuts

Basic editing

⌘X	Cut line (empty selection)
⌘C	Copy line (empty selection)
⇧ / ⇧	Move line down/up
⇧ / ⇧	Copy line down/up
⌘K	Delete line
⌘Enter / ⌘Enter	Insert line below/above
⌘J	Jump to matching bracket
⌘I / ⌘O	Indent/outdent line
Home / End	Go to beginning/end of line
⌘1 / ⌘1	Go to beginning/end of file
⇧PgUp / ⇧PgDn	Scroll line up/down
⇧PgUp / ⇧PgDn	Scroll page up/down
⇧ / ⇧	Fold/unfold region
⌘K ⌘I / ⌘K ⌘O	Fold/unfold all subregions
⌘K ⌘O / ⌘K ⌘J	Fold/unfold all regions
⌘K ⌘C	Add line comment
⌘K ⌘U	Remove line comment
⌘I	Toggle line comment
⇧A	Toggle block comment
⇧Z	Toggle word wrap

Multi-cursor and selection

⇧ + click	Insert cursor
⇧	Insert cursor above
⇧	Insert cursor below
⌘U	Undo last cursor operation
⇧I	Insert cursor at end of each line selected
⌘L	Select current line
⌘L	Select all occurrences of current selection
⌘F2	Select all occurrences of current word
⇧⇧ / ⇧	Expand / shrink selection
⇧ + drag mouse	Column (box) selection
⇧⇧ / ⇧	Column (box) selection up/down
⇧⇧ / ⇧	Column (box) selection left/right
⇧⇧PgUp	Column (box) selection page up
⇧⇧PgDn	Column (box) selection page down

Search and replace

⌘F	Find
⇧⇧	Replace
⌘G / ⌘G	Find next/previous
⇧Enter	Select all occurrences of Find match
⌘D	Add selection to next Find match
⌘K ⌘D	Move last selection to next Find match

Rich languages editing

⇧Space, ⌘I	Trigger suggestion
⇧Space	Trigger parameter hints
⇧F	Format document
⌘K ⌘F	Format selection
F12	Go to Definition
⇧F12	Peek Definition
⌘K F12	Open Definition to the side
⌘L	Quick Fix
⇧F12	Show References
F2	Rename Symbol
⌘K ⌘X	Trim trailing whitespace
⌘K M	Change file language

Navigation

⌘T	Show all Symbols
⇧G	Go to Line...
⌘P	Go to File...
⇧O	Go to Symbol...
⇧M	Show Problems panel
F8 / ⇧F8	Go to next/previous error or warning
⇧Tab	Navigate editor group history
⇧ / ⇧	Go back/forward
⇧M	Toggle Tab moves focus

Editor management

⌘W	Close editor
⌘K F	Close folder
⇧	Split editor
⇧1 / ⇧2 / ⇧3	Focus into 1 st , 2 nd , 3 rd editor group
⌘K ⇧ / ⌘K ⇧	Focus into previous/next editor group
⌘K ⇧ / ⌘K ⇧	Move editor left/right
⌘K ⇧ / ⌘K ⇧	Move active editor group

File management

⌘N	New File
⌘O	Open File...
⌘S	Save
⇧⇧	Save As...
⇧⇧	Save All
⌘W	Close
⌘K ⌘W	Close All

⇧T	Reopen closed editor
⌘K Enter	Keep preview mode editor open
⇧Tab / ⇧Tab	Open next / previous
⌘K P	Copy path of active file
⌘K R	Reveal active file in Finder
⌘K O	Show active file in new window/instance

Display

⇧⇧	Toggle full screen
⇧⇧	Toggle editor layout (horizontal/vertical)
⇧ / ⇧	Zoom in/out
⇧B	Toggle Sidebar visibility
⇧E	Show Explorer / Toggle focus
⇧F	Show Search
⇧G	Show Source Control
⇧D	Show Debug
⇧X	Show Extensions
⇧H	Replace in files
⇧J	Toggle Search details
⇧U	Show Output panel
⇧V	Open Markdown preview
⌘K V	Open Markdown preview to the side
⌘K Z	Zen Mode (Esc Esc to exit)

Debug

F9	Toggle breakpoint
F5	Start/Continue
F11 / ⇧F11	Step into/ out
F10	Step over
⇧F5	Stop
⌘K ⇧	Show hover

Integrated terminal

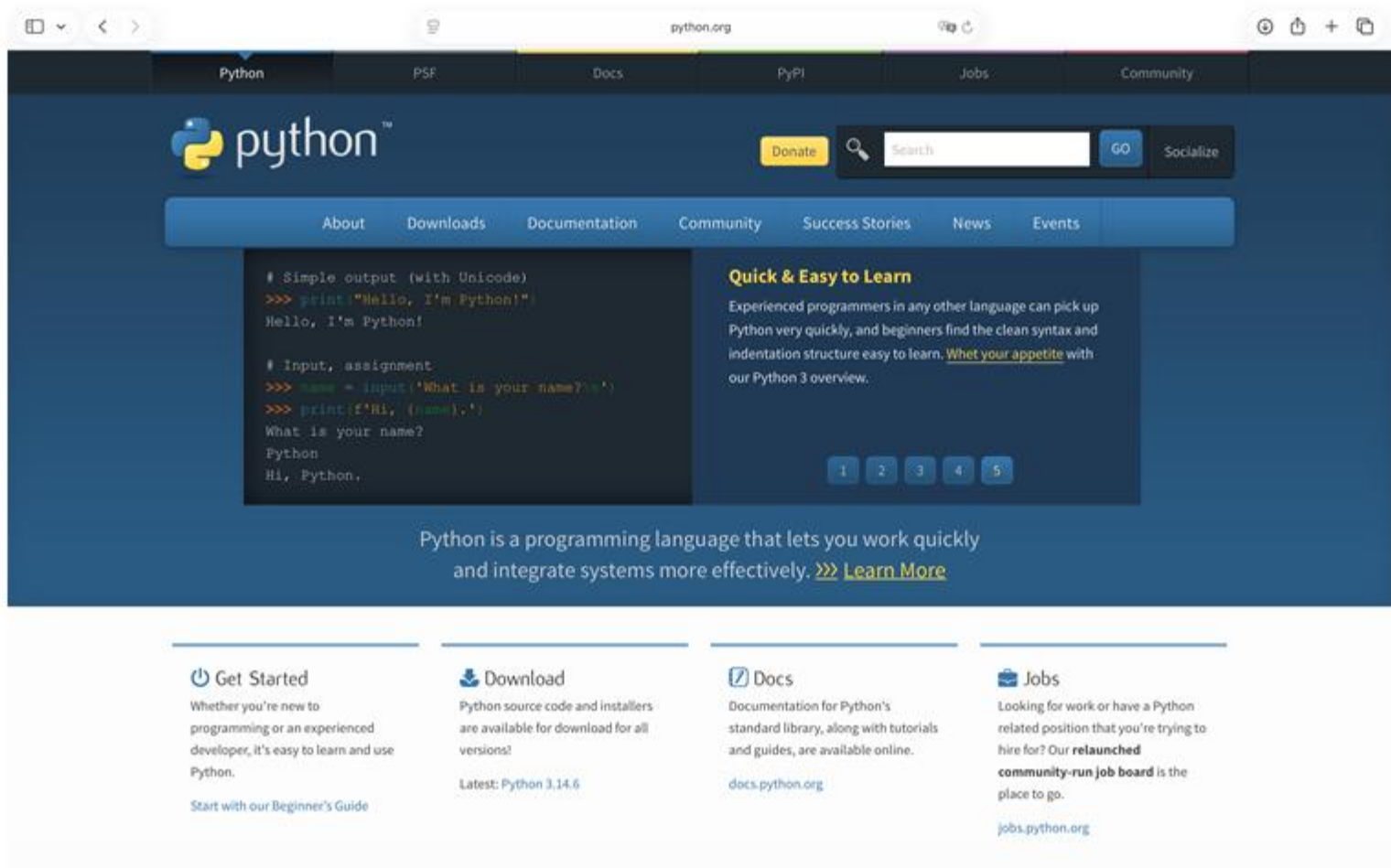
⇧	Show integrated terminal
⇧T	Create new terminal
⌘C	Copy selection
⇧ / ⇧	Scroll up/down
PgUp / PgDn	Scroll page up/down
⇧Home / End	Scroll to top/bottom

Other operating systems' keyboard shortcuts and additional unassigned shortcuts available at aka.ms/vscodekeybindings

Python 소개

개발 환경 구성

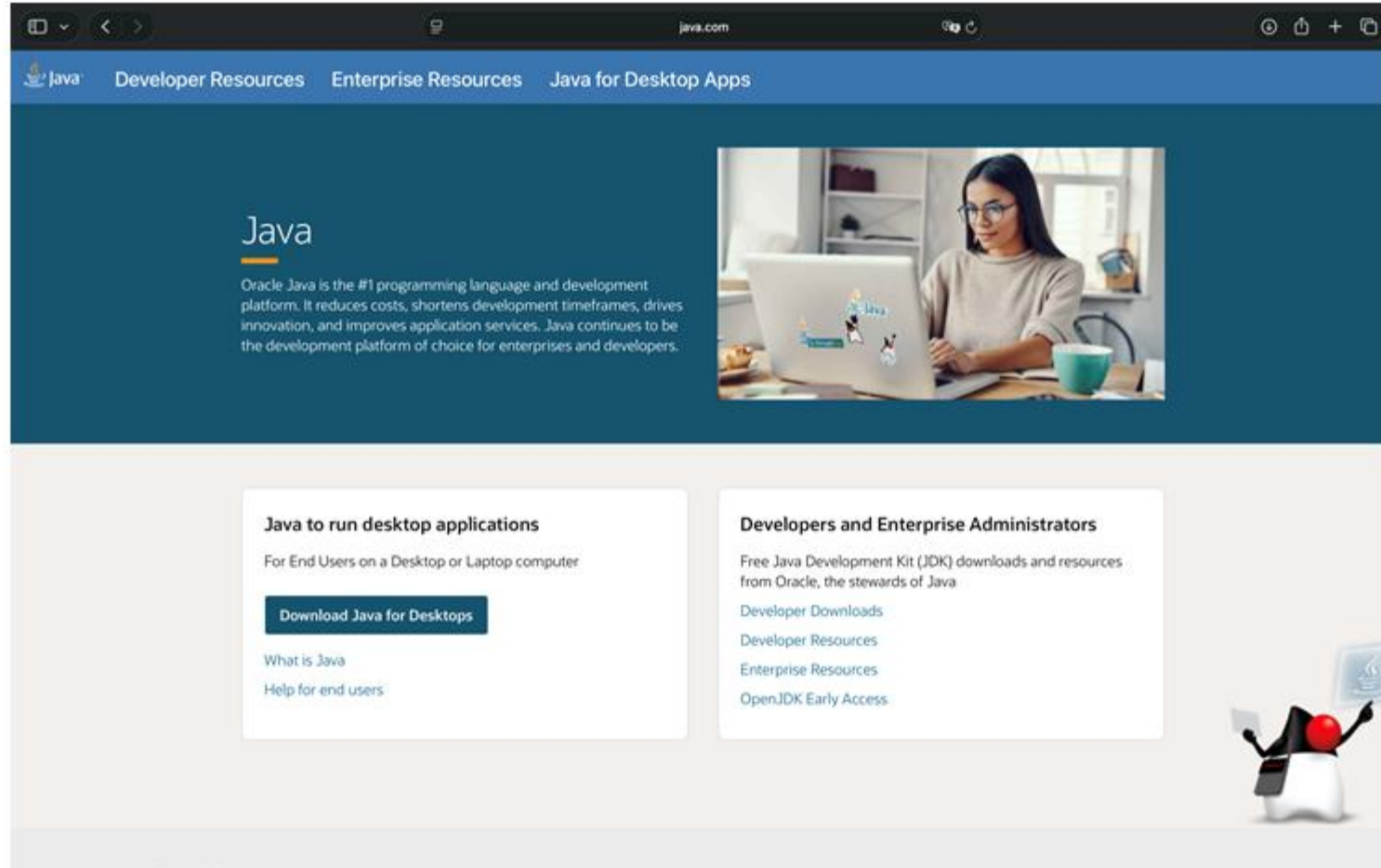
AI, Data 분야에 활발히 사용하는 프로그래밍 언어



Java 소개

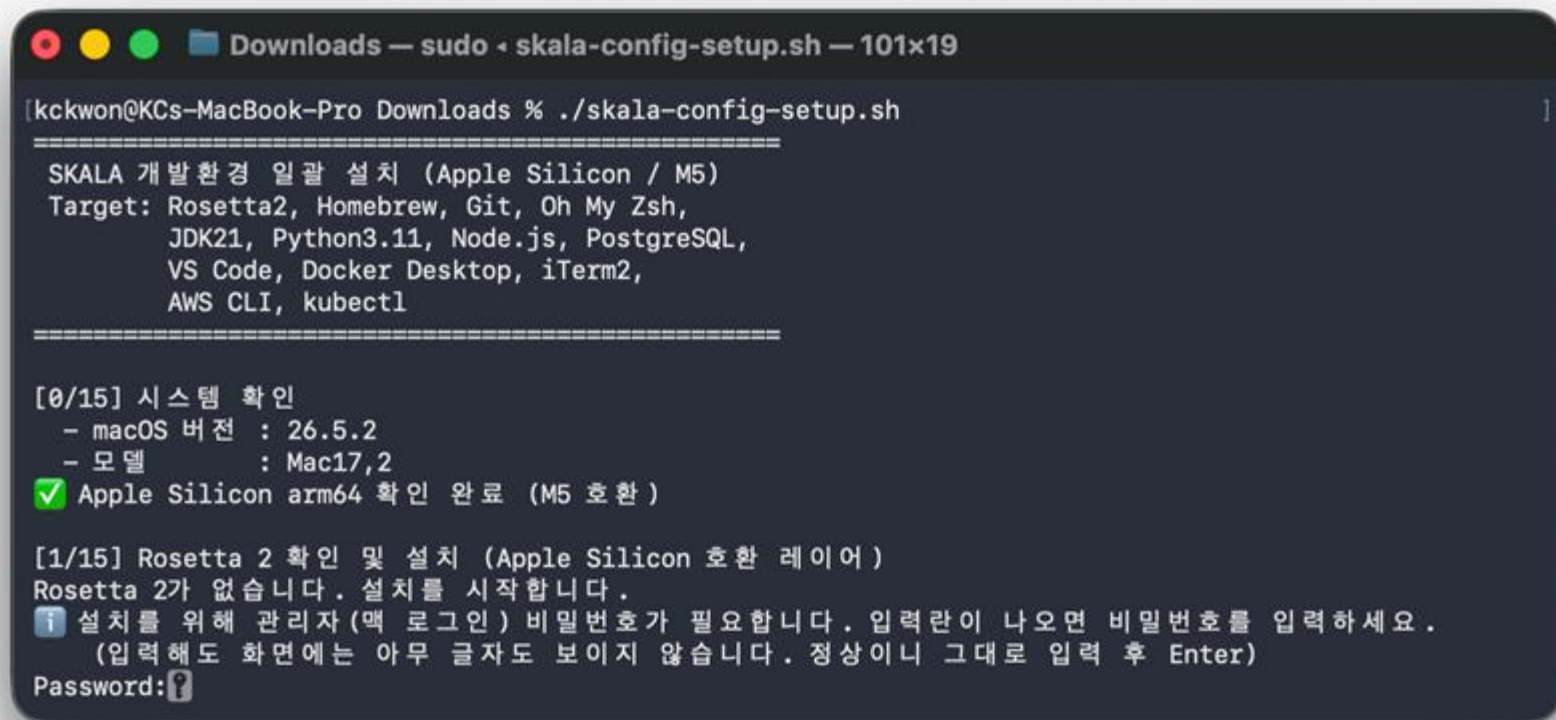
개발 환경 구성

백엔드, 엔터프라이즈 분야에 활발히 사용하는 프로그래밍 언어



개발 환경 구성을 위한 스크립트 실행

- scala-config-setup.sh 다운로드
- chmod u+x scala-config-setup.sh #실행 권한 추가
- ./scala-config-setup.sh #실행



```
Downloads — sudo < scala-config-setup.sh — 101x19

[kckwon@KCs-MacBook-Pro Downloads % ./scala-config-setup.sh

=====
SKALA 개발 환경 일괄 설치 (Apple Silicon / M5)
Target: Rosetta2, Homebrew, Git, Oh My Zsh,
        JDK21, Python3.11, Node.js, PostgreSQL,
        VS Code, Docker Desktop, iTerm2,
        AWS CLI, kubectl
=====

[0/15] 시스템 확인
- macOS 버전 : 26.5.2
- 모델      : Mac17,2
✅ Apple Silicon arm64 확인 완료 (M5 호환)

[1/15] Rosetta 2 확인 및 설치 (Apple Silicon 호환 레이어)
Rosetta 2가 없습니다. 설치를 시작합니다.
❗ 설치를 위해 관리자(맥 로그인) 비밀번호가 필요합니다. 입력란이 나오면 비밀번호를 입력하세요.
  (입력해도 화면에는 아무 글자도 보이지 않습니다. 정상이니 그대로 입력 후 Enter)
Password: 
```

개발 환경 종합 구성

개발 환경 구성

개발 환경 구성을 위한 스크립트 실행

설치 결과 점검

```
Downloads -- -zsh -- 101x50

[PostgreSQL]
psql (PostgreSQL) 17.18 (Homebrew)
postgresql@17 started kckwon ~/Library/LaunchAgents/homebrew.mxcl.postgresql@17.plist

[VS Code]
1.127.0
4fe60c8b1cdac1c4c174f2fb180d0d758272d713
arm64

[Docker]
Docker version 29.6.1, build 8900f1d
docker: unknown command: docker compose

Run 'docker --help' for more information

[iTerm2]
✅ iTerm2 설치됨 : /Applications/iTerm.app

[AWS CLI]
aws-cli/2.35.20 Python/3.14.6 Darwin/25.5.0 source/arm64

[kubectl]
Client Version: v1.36.2
Kustomize Version: v5.8.1

=====
종합 결과
=====
⚠️ 아래 1개 항목에서 문제가 발생했습니다 (나머지는 정상 진행됨):
❌ cask: visual-studio-code

👉 조치: 네트워크 상태를 확인한 뒤 스크립트를 다시 실행하세요.
이미 설치된 항목은 자동으로 건너뛰고, 실패 항목만 재시도합니다.

=====
설치 완료 - 마무리 안내
=====
1) 터미널을 완전히 종료 후 다시 열거나 아래 명령을 실행하세요.
source ~/.zprofile
source ~/.zshrc

2) Git 사용자 정보가 비어 있다면 아래 명령으로 설정하세요.
git config --global user.name "홍길동"
git config --global user.email "your-email@example.com"

3) Docker Desktop은 반드시 한 번 실행해야 docker 명령이 정상 동작합니다.
open -a Docker

=====
kckwon@KCs-MacBook-Pro Downloads %
```


Basics Mac Terminal Commands

Command	Description	Example
/ (Forward Slash)	Represents the root directory in the file system.	/
. (Single Period)	Refers to the current directory in which you're working.	.
.. (Double Period)	Refers to the parent directory (one level up from the current directory).	..
~ (Tilde)	Represents the home directory of the current user.	~
sudo [command]	Executes a command with elevated (super user) privileges.	sudo rm -rf /path/to/folder
nano [file]	Opens the Nano text editor to create or edit a file directly in the terminal.	nano myfile.txt
open [file]	Opens a specified file with the default application associated with its type.	open myfile.txt

Mac Terminal Command for Change Directory

Command	Description	Example
cd	Navigate to the home directory	cd
cd [folder]	Change to a specific directory (e.g., Documents, Downloads)	cd Documents
cd ~	Go to the home directory (shortcut for the user's home directory)	cd ~
cd /	Navigate to the root directory of the file system	cd /
cd -	Go back to the previous directory you were last working in	cd -
pwd	Print the current working directory	pwd
cd..	Move up one level to the parent directory	cd..
cd../..	Move up two levels in the directory structure	cd../..

List Directory Contents Commands

Command	Description	Example
ls	Lists all files and subdirectories in the current directory.	ls
ls -C	Displays the contents of the directory in a multi-column format.	ls -C
ls -a	Shows all entries in the directory, including hidden files (those starting with a period).	ls -a
ls -l	Lists files and directories, one entry per line.	ls -l
ls -F	Adds special symbols: a / after directories, a * after executable files, and an @ after symlinks.	ls -F
ls -S	Sorts files and directories by size, with the largest listed first.	ls -S
ls -l	Lists files in long format, including file permissions, owner, group, size, and modification date.	ls -l
ls -L /	Displays a detailed list of files starting from the root directory, including symbolic links.	ls -L /
ls -lt	Lists files in long format, sorted by modification time (newest first).	ls -lt
ls -lh	Displays file sizes in human-readable format (KB, MB, GB, etc.) along with other detailed information.	ls -lh
ls -lo	Lists files with detailed information, including file size, owner, and flags.	ls -lo
ls -la	Shows a detailed list of all files, including hidden files (those starting with a period).	ls -la

File and Directory Management Commands

Command	Description	Example
mkdir <dir>	Creates a new directory with the specified name.	mkdir Projects
mkdir -p <dir>/<dir>	Creates nested directories in a single command.	mkdir -p Work/2023/Reports
mkdir <dir1> <dir2> <dir3>	Creates multiple directories at once.	mkdir Folder1 Folder2 Folder3
mkdir "<dir>"	Creates a folder with spaces in its name.	mkdir "My New Folder"
rmdir <dir>	Deletes an empty directory.	rmdir OldFolder
rm -R <dir>	Removes a directory and all of its contents.	rm -R ProjectFolder
touch <file>	Creates a new, empty file without any extension.	touch newfile.txt
cp <file> <dir>	Copies a file to a specified directory.	cp file.txt /Documents/
cp <file> <newfile>	Copies a file to the current directory and renames it.	cp file.txt newfile.txt
cp <file>~/<dir>/<newfile>	Copies a file to a specified folder and renames it during the process.	cp report.txt ~/Documents/Reports/summary.txt
cp -R <dir> <"new dir">	Copies an entire directory, including its contents, to a new location.	cp -R folder1 "New Folder"

cp -i <file><dir>	Prompts you for confirmation before overwriting a file during copy.	cp -i file1.txt /Backup/
cp <file1> <file2> <file3>/Users/<dir>	Copies multiple files into a directory.	cp file1.txt file2.txt /Users/username/Documents/
ditto -V [folder path][new folder]	Copies the contents of one folder to another, with status updates.	ditto -V /Folder1 /Folder2
rm <file>	Deletes a specified file permanently.	rm unwantedfile.txt
rm -i <file>	Deletes a file with a prompt for confirmation.	rm -i oldfile.txt
rm -f <file>	Forces the deletion of a file without any confirmation.	rm -f tempfile.txt
rm <file1> <file2> <file3>	Deletes multiple files at once without confirmation.	rm file1.txt file2.txt file3.txt
mv <file> <newfilename>	Moves or renames a file to a new location or name.	mv oldfile.txt newfile.txt
mv <file> <dir>	Moves a file to a different directory.	mv file.txt /Documents/Backup/
mv -i <file> <dir>	Moves a file to a folder and asks for confirmation before overwriting.	mv -i file1.txt /Documents/Backup/
mv *.png ~/<dir>	Moves all PNG files from the current directory to another directory.	mv *.png ~/Pictures/

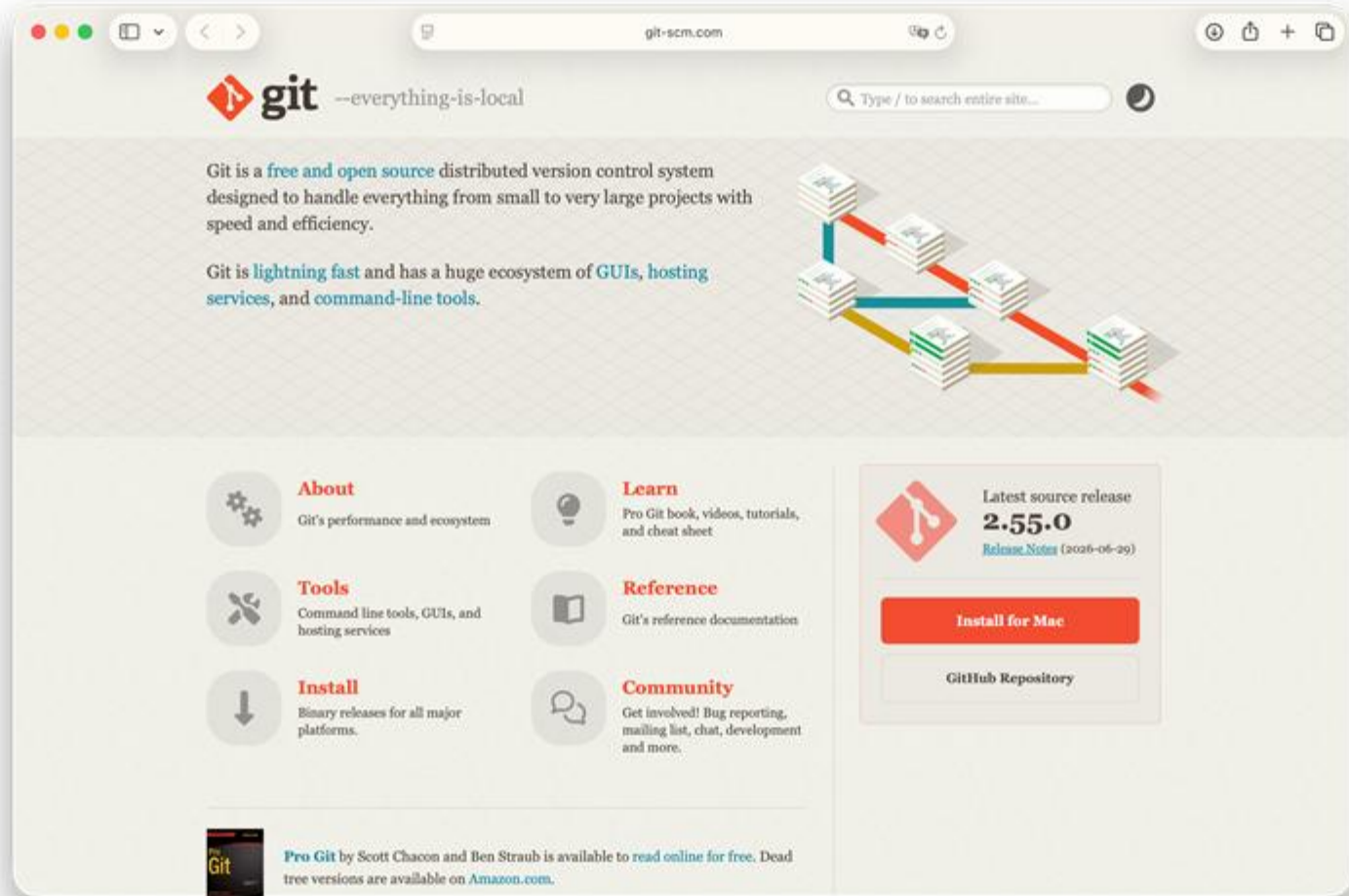
3. Git

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Git 소개

Git

Git은 DVCS(Distributed Version Control System)



저장소(Repository)

파일들과 변경 이력(History)을 기록

파일들과 그 파일들이 언제, 어떻게, 누구에 의해 변경되었는지에 대한 모든 변경 이력(History)이 기록되는 가상의 보관소

로컬 저장소 (Local Repository)

- 내 컴퓨터에 존재하는 저장소
- 네트워크 연결이 끊긴 오프라인 상태(비행기나 기차 안)에서도 로컬 저장소가 있다면 코드 수정 후 버전을 기록(커밋)하고 히스토리를 조회하는 모든 작업이 가능

원격 저장소 (Remote Repository)

- 인터넷이나 네트워크 어딘가에 있는 외부 저장소 (예: GitHub)
- 동료들과 협업할 때 원격 저장소를 사용
- 각자 로컬 저장소에서 작업한 내용을 원격 저장소에 push(밀어넣기)하고, 동료가 작업한 내용을 pull(당겨오기)하면서 프로젝트를 함께 완성

저장소(Repository)

파일들과 변경 이력(History)을 기록

파일들과 그 파일들이 언제, 어떻게, 누구에 의해 변경되었는지에 대한 모든 변경 이력(History)이 기록되는 가상의 보관소

작업 디렉토리 (Working Directory)

- 실제 눈에 보이는 파일들이 있는 컴퓨터 디렉토리
- 사용자가 코드를 수정하고 편집하는 작업실

스테이징 영역 (Staging Area)

- 작업 디렉토리에서 수정한 파일 중, 다음 버전(커밋)에 포함할 파일들을 임시로 올려두는 준비 무대

Git 디렉토리 (.git Directory)

- git commit을 완료하면 스냅샷이 영구적으로 저장되는 저장소의 본체(핵심 데이터베이스)

브랜치(Branch)

Git

원래 코드와는 상관없이 독립적으로 개발을 진행할 수 있도록 해주는 갈라져 나온 나뭇가지(분기선)

파일들과 그 파일들이 언제, 어떻게, 누구에 의해 변경되었는지에 대한 모든 변경 이력(History)이 기록되는 가상의 보관소

브랜치(Branch)

- Git은 변경 사항(Diff)이 아니라 파일의 스냅샷(Snapshot)을 기록
- 커밋(git commit)을 할 때마다 새로운 스냅샷이 생성
- 커밋은 이전 부모 커밋이 무엇인지 정보를 가지고 있음
- 단순히 특정 커밋 개체를 가리키는 '40글자짜리 SHA-1 체크섬 파일(포인터)'에 불과

HEAD

- 현재 작업 중인 로컬 브랜치를 가리킴
- 브랜치를 이동하는 명령인 git checkout 이나 git switch를 실행하면, HEAD가 가리키는 포인터가 해당 브랜치로 옮겨가고 내 컴퓨터의 워킹 디렉토리 파일들도 그 브랜치의 마지막 커밋 시점으로 되돌아감

브랜치(Branch)

Git

원래 코드와는 상관없이 독립적으로 개발을 진행할 수 있도록 해주는 갈라져 나온 나뭇가지(분기선)

파일들과 그 파일들이 언제, 어떻게, 누구에 의해 변경되었는지에 대한 모든 변경 이력(History)이 기록되는 가상의 보관소

리모트 브랜치 (Remote Branch)

- 인터넷이나 네트워크 어딘가에 있는 원격 저장소(GitHub 등)의 브랜치 포인터
- 원격 저장소에 마지막으로 연결했던 순간, 그 서버의 브랜치가 어디를 가리키고 있었는지 나타내는 '북마크' 역할
- 서버와 통신(git fetch 등)할 때만 자동으로 갱신

생성하기와 가져오기

git init

새로운 로컬 Git 저장소(Repository)를 생성. 저장소 디렉토리 내에 .git 디렉토리가 생성되며 버전 관리가 시작

```
git init
```

git clone

원격 서버에 이미 존재하는 기존 저장소를 로컬 컴퓨터로 복사.
단순 소스코드 다운로드가 아니라 전체 커밋 히스토리를 함께 받아옴.

```
git clone https://github.com/user/repo.git
```

스냅샷 다루기

`git status`

현재 작업 디렉토리(Working Directory)와 스테이징 영역(Staging Area)의 상태를 확인

```
git status
```

```
# 좀 더 짧고 간결하게 상태를 확인하고 싶을 때  
git status -s
```

`git add`

작업 디렉토리의 변경 내용(새 파일, 수정된 파일)을 다음 커밋에 포함하기 위해 스테이징 영역에 추가

```
# 특정 파일만 추가할 때  
git add README.md
```

```
# 변경되거나 새로 생성된 모든 파일을 추가할 때  
git add .
```


스냅샷 다루기(계속)

`git commit`

스테이징 영역에 올라온 파일들의 스냅샷을 로컬 저장소에 영구적으로 기록(커밋)

```
# 인라인으로 메시지를 작성하여 커밋할 때  
git commit -m "Initial commit message"
```

```
# 이미 추적 중인(Tracked) 파일의 수정을 add 과정 없이 바로 커밋할 때  
git commit -am "Fix benchmarks for speed"
```

변경 사항 비교하기

`git diff`

두 트리 간의 차이점을 비교.

옵션이 없으면 현재 워킹 디렉토리와 스테이징 영역을 비교, 옵션을 주면 스테이징된 변경 사항 또는 커밋 간의 차이를 비교.

```
# 아직 스테이징되지 않은(Unstaged) 변경 사항을 볼 때  
git diff
```

```
# 이미 스테이징된(Staged) 변경 사항을 볼 때  
git diff --staged
```

브랜치와 병합

git branch

브랜치를 관리하는 명령어.

아무런 옵션 없이 실행하면 현재 저장소에 있는 브랜치 목록을 보여주며, 브랜치를 새로 생성하거나 삭제할 때도 사용

브랜치 목록 조회

git branch

'banana'라는 이름의 새 브랜치 생성

git branch banana

'banana' 브랜치 삭제

git branch -d banana

브랜치와 병합(계속)

git switch

현재 작업 중인 브랜치를 변경하거나, 특정 브랜치로 이동

```
# 'testing' 브랜치로 이동할 때  
git switch testing
```

```
# 새 브랜치를 만들고 동시에 해당 브랜치로 바로 이동할 때  
git switch -c new-feature
```

git merge

지정한 다른 브랜치의 변경 사항을 현재 내가 체크아웃하고 있는 브랜치로 가져와 병합

```
# 'banana' 브랜치의 작업 내용을 현재 브랜치로 병합할 때  
git merge banana
```

공유하고 업데이트하기

git fetch

원격 저장소(Remote)의 최신 변경 사항과 데이터를 로컬 저장소로 모두 가져옴

```
git fetch origin
```

git pull

원격 저장소의 변경 사항을 가져온 후(fetch), 현재 로컬 브랜치에 자동으로 병합(merge)

```
git pull origin main
```

공유하고 업데이트하기(계속)

```
git push
```

로컬 저장소에서 작업하고 커밋한 변경 사항들을 원격 저장소(예: GitHub)로 전송하여 업데이트

```
# 로컬의 main 브랜치를 원격(origin) 저장소로 보낼 때  
git push origin main
```

설정

git config

정보를 설정

```
#사용자 이름 설정  
git config --global user.name "홍길동"
```


도움말

git help

도움말

git 사용법
git help

git 명령어 사용법
git help 명령어

```
kckwon@KCs-MacBook-Pro ~ % git help
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
       [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
       [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
       [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
       [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
       <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
  bisect     Use binary search to find the commit that introduced a bug
  diff       Show changes between commits, commit and working tree, etc
  grep       Print lines matching a pattern
  log        Show commit logs
  show       Show various types of objects
  status     Show the working tree status

grow, mark and tweak your common history
  backfill   Download missing objects in a partial clone
  branch     List, create, or delete branches
  commit     Record changes to the repository
  merge      Join two or more development histories together
  rebase     Reapply commits on top of another base tip
  reset      Reset current HEAD to the specified state
  switch     Switch branches
  tag        Create, list, delete or verify a tag object signed with GPG

collaborate (see also: git help workflows)
  fetch      Download objects and refs from another repository
  pull       Fetch from and integrate with another repository or a local branch
  push       Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.
kckwon@KCs-MacBook-Pro ~ %
```

4. GitHub

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

GitHub 소개

Github

Git을 기반으로 하여 공동 작업 도구, 자동화 기능 및 사용자에게 친숙한 웹 인터페이스를 추가한 플랫폼



[실습] GitHub 계정 생성

GitHub

GitHub 가입하여 계정을 생성

Private

github.com

Already have an account? [Sign in](#)

Create your free account

Explore GitHub's core features for individuals and organizations.

[See what's included](#)



Sign up for GitHub

Continue with Google

Continue with Apple

or

Email

Password

Username

Your Country/Region

☒ Sign up for Copilot Free for coding assistance on the web, in the IDE, and in the CLI. [See terms](#)

Create account

By creating an account, you agree to the [Terms of Service](#). For more information about GitHub's privacy practices, see the [GitHub Privacy Statement](#). We'll occasionally send you account-related emails.

[실습] Repository 생성

Github

Github에서 Repository 생성

New repository

github.com

Type to search

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner *
kckwon /

Repository name *

Great repository names are short and memorable. How about [studious-guide](#)?

Description

0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository
Public

Start with a template
Templates pre-configure your repository with files.

No template

Add README
READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license
Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

45

[실습] Repository 생성

Github

Repository 생성을 위한 값 설정

github.com

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1

General

Owner *

kckwon

Repository name *

skala_git_example

skala_git_example is available.

Great repository names are short and memorable. How about [studious-guide](#)?

Description

SKALA Git Example Repository

28 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Start with a template

Templates pre-configure your repository with files.

No template

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

On

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

GitHubPages

Add license

Licenses explain how others can use your code. [About licenses](#)

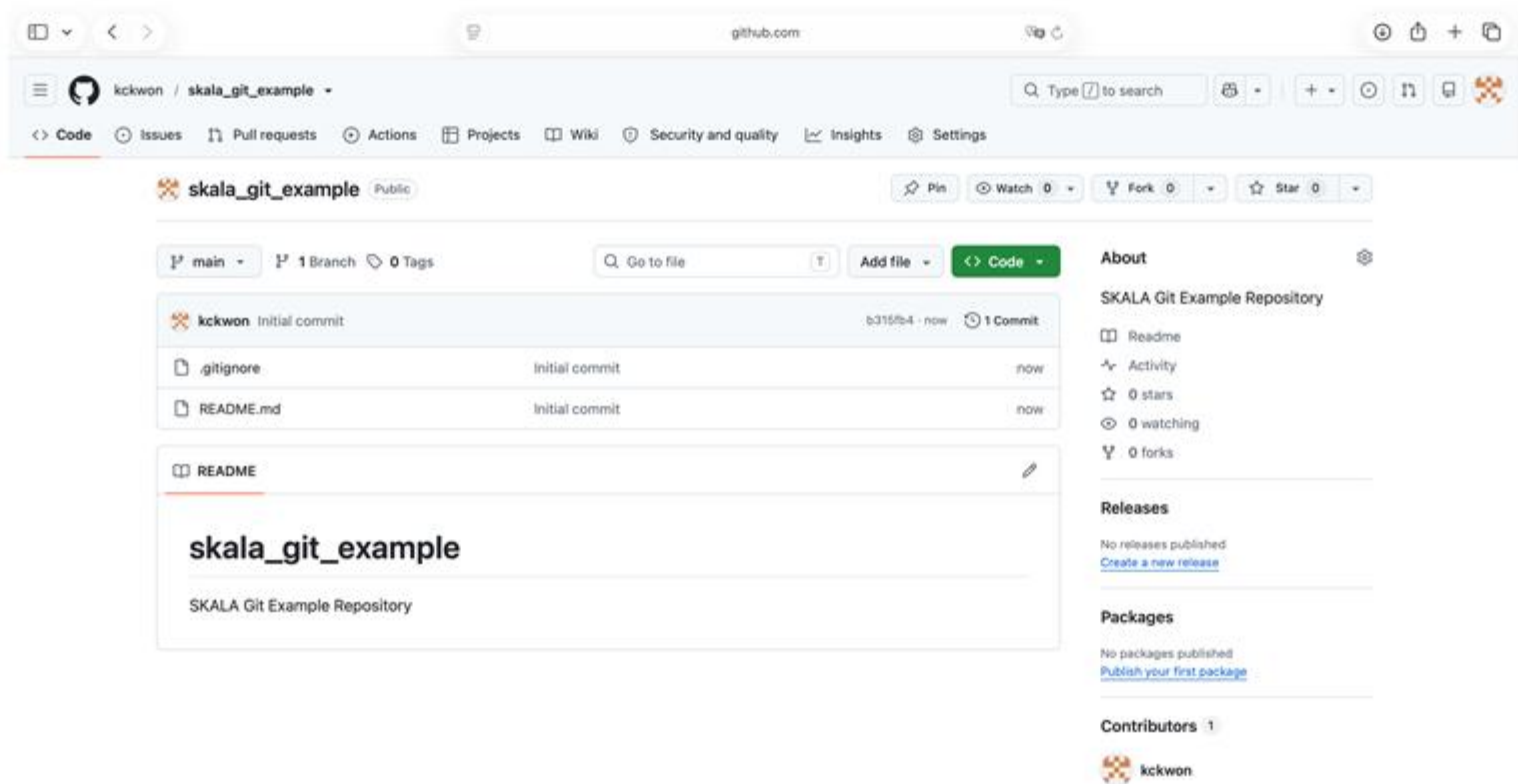
No license

Create repository

[실습] Repository 생성

Github

생성된 Repository 확인



GitHub is a collaborative platform

Github

GitHub issue는 GitHub에서 작업에 대한 아이디어, 피드백, 작업 또는 버그를 추적하는 데 사용됨

이슈(Issues)

- 이슈는 작업에 대한 아이디어, 피드백, 작업 또는 버그를 추적하는 데 사용

Discussions(토론)

- 토론은 Q&A, 아이디어 또는 일반적인 피드백과 같이 코드와 반드시 관련되지 않은 여러 대화를 위해 설계됨
- 공유 포럼 내에서 개방적이고 지속적인 의사소통을 지원하며 저장소의 가시성에 따라 공개 또는 비공개일 수 있음

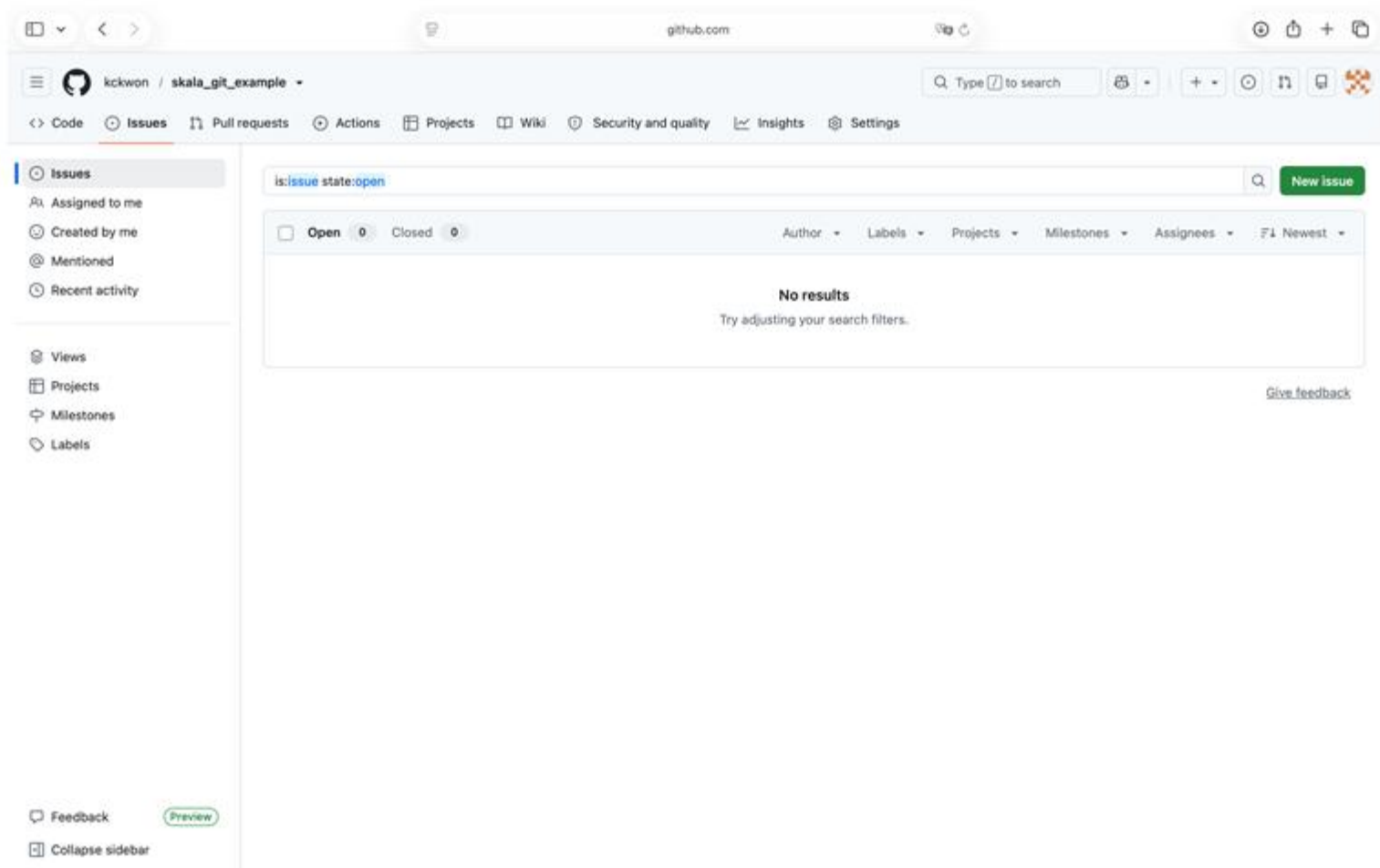
GitHub Pages

- GitHub.com 저장소에서 직접 자신, 조직 또는 프로젝트에 대한 웹 사이트를 공개
- GitHub의 저장소에서 HTML, CSS 및 JavaScript 파일을 바로 사용하는 정적 사이트 호스팅 서비스
- 필요에 따라 빌드 프로세스를 통해 파일을 실행하고 웹 사이트를 게시

[실습] Issue 생성

Github

Issue 메인 화면



[실습] Issue 생성

Github

Issue 생성 화면

The screenshot shows the GitHub interface for creating a new issue. The browser address bar shows 'github.com'. The repository path is 'kckwon / skala_git_example'. The navigation bar includes links for Code, Issues (selected), Pull requests, Actions, Projects, Wiki, Security and quality, Insights, and Settings. The main content area is titled 'Create new issue'. It features a text input field for the title, a rich text editor for the description (with 'Write' and 'Preview' tabs), and a right-hand sidebar for assigning the issue to a user, adding labels, projects, or milestones. At the bottom, there are options to 'Create more', 'Cancel', or 'Create' the issue.

github.com

kckwon / skala_git_example

Type to search

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Create new issue

Add a title *

Title

Add a description

Write Preview

Type your description here...

Assignees

No one - [Assign yourself](#)

Labels

No labels

Projects

No projects

Milestone

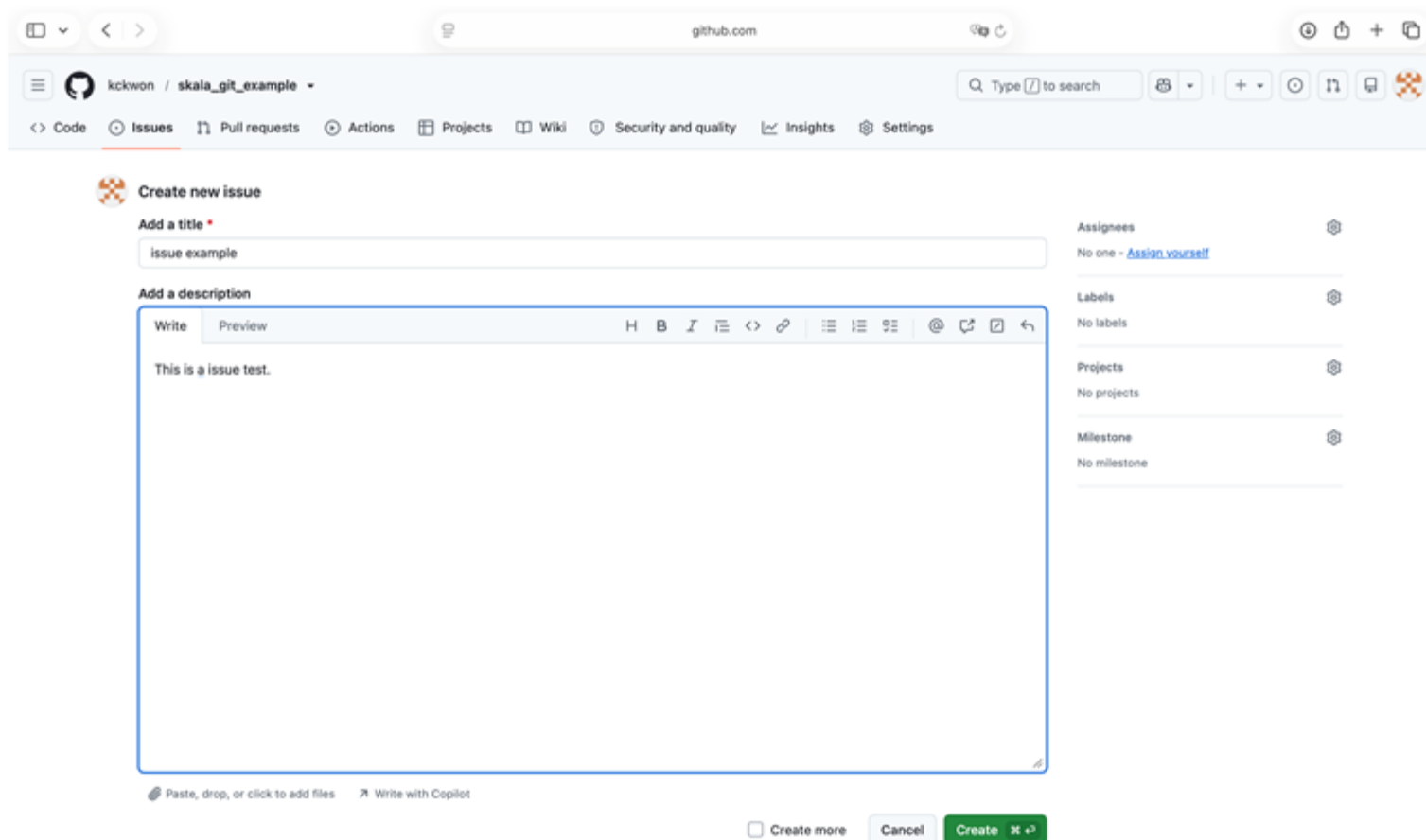
No milestone

Paste, drop, or click to add files Write with Copilot

Create more Cancel Create

[실습] Issue 생성

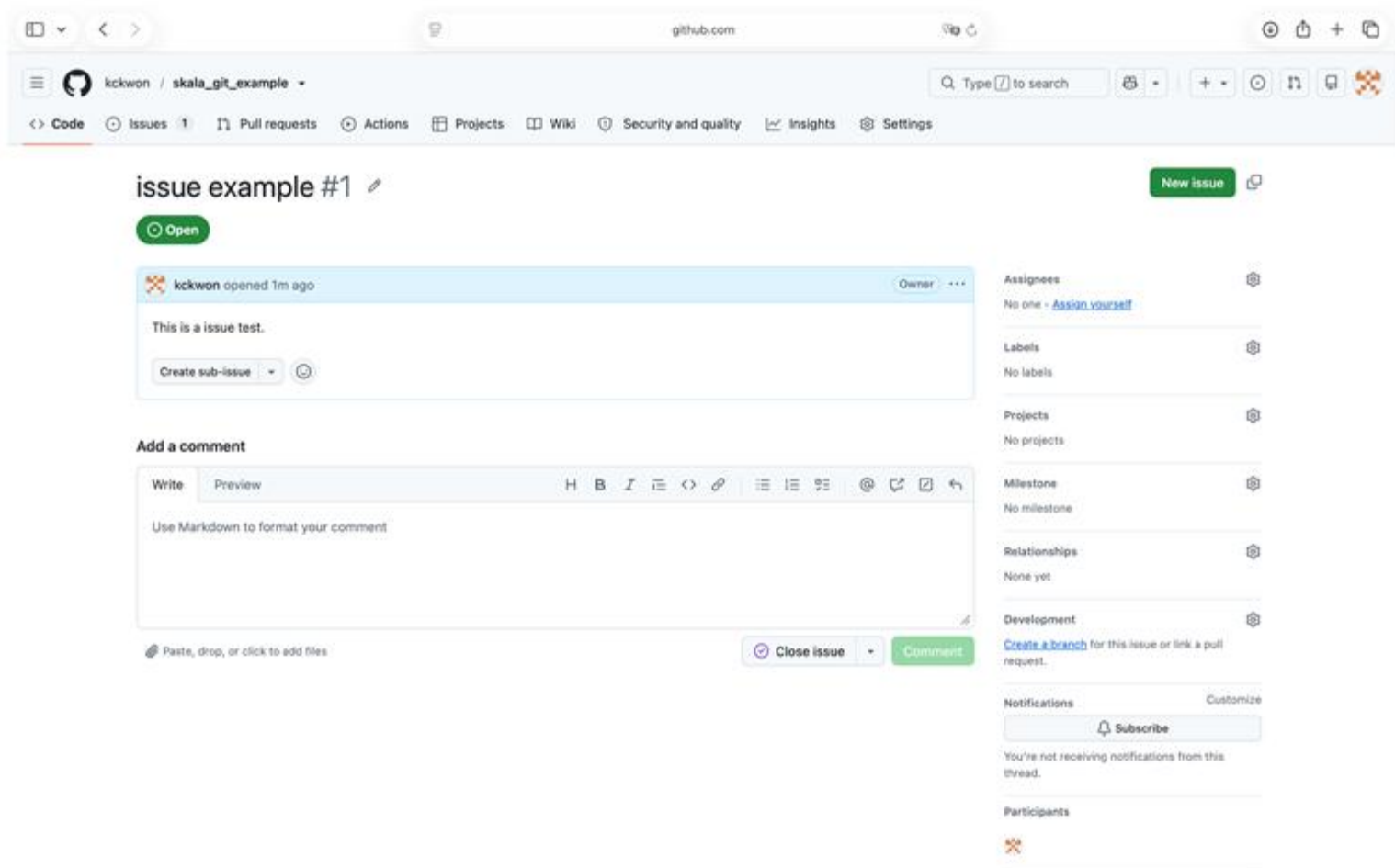
Issue 생성 화면에서 값 세팅



[실습] Issue 생성

Github

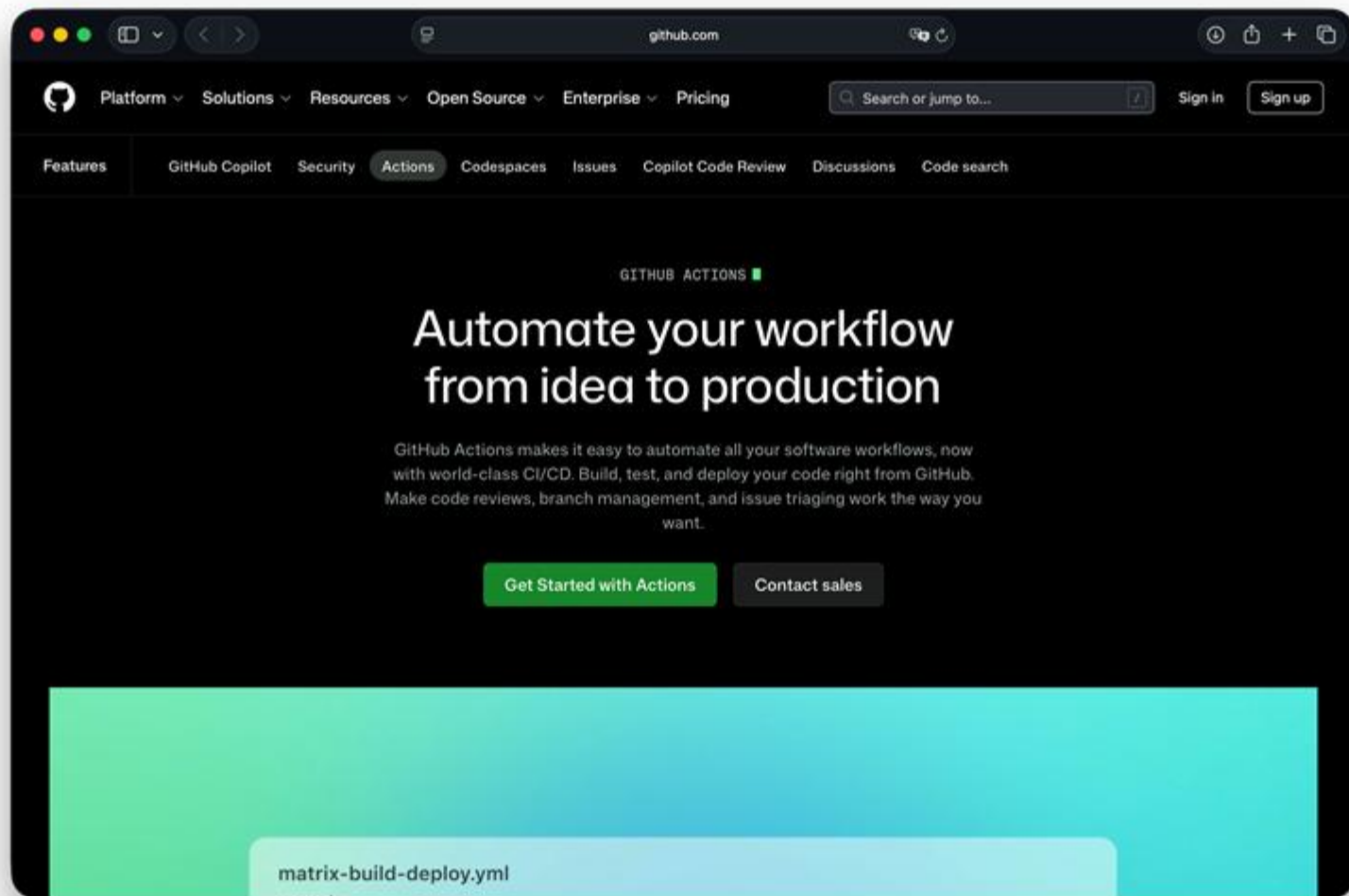
Issue 화면에서 값 확인



GitHub Action

Github

GitHub Action은 Github로부터 CI/CD workflow를 자동화



GitHub Action은 Github로부터 CI/CD workflow를 자동화

- GitHub Actions는 빌드, 테스트 및 배포 파이프라인을 자동화할 수 있는 CI/CD(연속 통합 및 지속적인 업데이트) 플랫폼
- 저장소에 대한 모든 끌어오기(pull request) 요청을 빌드하고 테스트하는 워크플로를 만들거나 병합된 끌어오기 요청을 프로덕션에 배포
- GitHub Actions에서는 DevOps를 넘어 저장소에서 다른 이벤트가 발생할 때 워크플로를 실행
- 누군가가 리포지토리에서 새 이슈를 만들 때마다 워크플로를 실행하여 적절한 레이블을 자동으로 추가

참고자료

강의에 사용한 참고자료 목록

[Computer Dictionary, Terms, and Glossary](#)

[Programming Tutorial | Introduction, Basic Concepts, Getting started, Problems - GeeksforGeeks](#)

[keyboard-shortcuts-macos.pdf](#)

[Complete Mac Terminal Commands Cheat Sheet - GeeksforGeeks](#)

[GitHub란? - Training | Microsoft Learn](#)

[GitHub는 협업 플랫폼임 - Training | Microsoft Learn](#)

[GitHub 플랫폼 관리 - Training | Microsoft Learn](#)

[GitHub Actions](#)

[GitHub Actions에 대한 이해 - GitHub 문서](#)

SKALA를 시작합니다!

종합 실습 진행